Como medir os seus agentes de IA

Do "parece que melhorou" para a medida: como saber se uma mudança no seu agente de código ajudou de verdade, com tarefas, um teste que o agente não vê, repetição e margem de erro. E como rodar e adaptar o repositório agent-evaluation ao seu projeto.

---

1. Por que medir
2. O que é "efetivo"
3. Montando as tarefas
4. Rodando o experimento
5. Lendo o resultado
6. Melhorando o agente
7. Caso real: agentes KTLO na Creditas
8. Armadilhas
9. Glossário
10. Referências

Fonte: https://github.com/MateusCastro2203/agent-evaluation

Por que medir

Este ebook acompanha a palestra "Como medir os seus agentes de IA". A palestra tem 40 minutos, com as perguntas. Aqui dá para ir com calma, ver o código e voltar quando for montar a sua própria medição.

Você não precisa saber estatística nem ter rodado uma eval antes. Os capítulos para quem está começando usam palavras do dia a dia. Quando aparece um termo técnico, ele vem entre parênteses na primeira vez: é o termo que você vai digitar no buscador se quiser ir mais fundo. O glossário, no fim, junta todos.

"Parece que melhorou"

Um **agente de código** é um programa que usa um modelo de IA para mexer no seu projeto: ele lê arquivos, edita, roda os testes e decide o próximo passo sozinho. O Claude Code, que a palestra usa, é um deles: roda no terminal e trabalha no seu repositório.

Todo mundo que usa um agente assim já fez isto:

1. mudou alguma coisa: o prompt, o modelo ou o `CLAUDE.md` (um arquivo de texto com as regras da casa, que o Claude Code lê antes de começar);
2. pediu **uma** tarefa;
3. deu certo;
4. "melhorou!".

Esse "avaliar no olho" tem nome: *vibe check*. Ele não é inútil. É como você descobre que vale a pena olhar para alguma coisa. O problema é parar nele, porque muita coisa ficou de fora:

- A tarefa era fácil? Uma tarefa fácil dá certo com qualquer configuração.
- Se você pedisse de novo, daria certo de novo?
- Quanto custou? Quantas idas e vindas o agente deu?
- Ele mexeu onde não devia?
- Teria dado certo **antes** da mudança também?

A última pergunta é a mais esquecida. Sem rodar a configuração antiga nas mesmas tarefas, você não tem com o que comparar.

O que é uma eval

Eval vem de *evaluation*, avaliação. Em uma frase: um conjunto de tarefas em que "deu certo" é conferido por código, repetidas várias vezes, para comparar o antes e o depois de uma mudança.

São três ingredientes. Cada um tem uma analogia.

1. "Deu certo" conferido por código. Pense numa prova com gabarito. O aluno recebe o enunciado, mas não vê o gabarito, que fica com o professor. Aqui, o enunciado é o pedido ao agente, escrito como uma issue (o pedido de trabalho que se abre no GitHub ou no Jira). O gabarito é um teste automático: um pequeno programa que confere se o código faz o que devia. O agente nunca vê esse teste. Por isso a palestra o chama de **teste escondido**.

2. Repetidas várias vezes. O agente não faz a mesma coisa duas vezes (próxima seção). Jogue uma moeda 4 vezes e tire 3 caras: isso não prova que a moeda é viciada. Jogue 400 vezes e tire 300: agora prova. Com o agente é igual. Uma execução que deu certo diz pouco. Muitas execuções dizem bastante.

3. Antes × depois de uma mudança. Você roda as mesmas tarefas com a configuração antiga e com a nova, e compara. Neste ebook, cada versão comparada se chama **variante**: a A é o "antes", a B é o "depois".

O agente não faz a mesma coisa duas vezes

Mesmo pedido, mesmo repositório, mesmo modelo: o resultado muda de uma execução para outra. O modelo escreve o texto um pedacinho por vez. Cada pedacinho se chama **token** e é sorteado entre as opções mais prováveis. Uma diferença pequena no começo vira uma diferença grande no fim. Um agente lê o arquivo A primeiro, o outro lê o B; um roda os testes, o outro não. No fim, um dá certo e o outro não.

Isso é o comportamento normal da ferramenta, não um defeito. No jargão, o agente **não é determinístico**.

A consequência prática: **uma execução é uma amostra, não um resultado**. A pergunta certa não é "deu certo?". É "com que frequência dá certo?".

Margem de erro: como numa pesquisa eleitoral

Pesquisa eleitoral não diz só "o candidato tem 40%". Ela diz "40%, com margem de erro de 3 pontos". Com poucas pessoas ouvidas, a margem é larga. Com mais pessoas, ela estreita.

A taxa de acerto de um agente funciona do mesmo jeito. O repositório calcula a margem de erro com uma conta chamada **intervalo de Wilson**, com 95% de confiança (o capítulo 5 explica). Dois números para calibrar a intuição:

O que você viu	Onde a taxa real pode estar (margem de erro)
deu certo 1 de 1 vez	entre **21% e 100%**
deu certo 16 de 16 vezes	entre **81% e 100%**

Uma execução que deu certo combina até com um agente que acerta uma vez a cada cinco. E mesmo 16 acertos em 16 tentativas não provam 100%. Esses números são conta, não opinião: é a mesma fórmula que o relatório do repositório usa.

O experimento da palestra

Eu escrevi um `CLAUDE.md` para um app pequeno de exemplo, com as regras da casa: que tipo de erro lançar, como lidar com dinheiro, de onde vem a hora atual, que pasta não se toca. Acho que ele ajuda o agente. Mas "acho" não é medida. Então, no começo da palestra, eu rodo `pnpm eval`, e o Claude Code faz as mesmas 8 tarefas **sem** o `CLAUDE.md` (variante A) e **com** ele (variante B), duas vezes cada: 32 execuções, enquanto eu falo. No fim, o próprio comando gera um relatório que diz "Com CLAUDE.md foi melhor", "Sem CLAUDE.md foi melhor" ou "Ainda não dá para saber". No ensaio, deu "Ainda não dá para saber", e o capítulo 5 mostra por quê.

O repositório `agent-evaluation` roda esse mesmo experimento no seu computador, com o login normal do Claude Code. Não precisa de chave de API.

Como ler este ebook

- **Está começando?** Leia os capítulos 1, 2, 5 e 8. Eles explicam o que medir, como ler o resultado e onde é fácil se enganar. O glossário, no fim, ajuda com os termos.
- **Vai rodar e adaptar o repositório?** Leia também os capítulos 3, 4 e 6. Eles mostram como as tarefas são montadas, o que acontece em cada execução e como trocar o app de exemplo pelo seu projeto.
- **O capítulo 7** traz lições de agentes em produção. Vale para os dois públicos.

Nos capítulos mais técnicos, as partes avançadas ficam em seções marcadas como **"Para quem quer ir fundo"**. Dá para pular sem perder o fio.

O que você leva deste ebook

Ao final, você consegue:

- dizer o que medir num agente de código: se deu certo, quanto custou, quantas voltas deu, quanto demorou, se mexeu onde não devia e se resolve sempre (capítulo 2);
- montar um conjunto pequeno de tarefas com testes escondidos e rodar o seu agente contra ele (capítulos 3 e 4);
- comparar duas configurações, por exemplo sem e com `CLAUDE.md`, e ler o resultado com a margem de erro, sem se enganar com uma execução só (capítulo 5);
- transformar uma falha do dia a dia em tarefa nova da eval (capítulo 6).

O que é "efetivo"

"O agente está bom?" não tem resposta enquanto "bom" não estiver definido. Para um agente de código, "bom" são seis números, não um.

As seis métricas

O que medir	Pergunta	Como o repositório mede
Sucesso	deu certo?	o teste escondido da tarefa passou (o critério de feito)
Custo	quanto gastou?	tokens e custo estimado, que o Claude Code informa no fim
Voltas (turnos)	quantas idas e vindas deu?	o número de turnos que o Claude Code informa
Tempo	quanto demorou?	a duração que o Claude Code informa
Mexeu onde não devia (fora do escopo)	alterou arquivo que a tarefa não permitia?	arquivos alterados fora da lista `permitidos` da tarefa
Resolve sempre? (consistência, pass^k)	acerta em todas as tentativas?	a tarefa deu certo em todas as repetições

Algumas definições:

- **Critério de feito:** a regra objetiva que diz se a tarefa foi resolvida. Aqui é sempre um teste automático que o agente não vê, o teste escondido (capítulo 3).
- **Voltas (turnos):** uma volta é, mais ou menos, uma ida e volta entre o modelo e uma ferramenta: ler um arquivo, editar, rodar os testes. Mais voltas costumam significar mais tokens e mais tempo.
- **Token:** o pedacinho de texto que o modelo lê ou escreve. É a unidade de consumo. Tokens de entrada (o que o modelo lê), de saída (o que ele escreve) e de cache (texto que ele já tinha lido e reaproveita) são contados separados.

Por que "deu certo" sozinho engana

Dois agentes com a mesma taxa de acerto podem ser muito diferentes. Um pode gastar o dobro de voltas. Outro pode resolver a tarefa mexendo num arquivo que ninguém podia tocar. Se você olhar só a taxa, os dois parecem iguais.

Por isso "mexeu onde não devia" é uma métrica separada, de propósito. "Resolveu, mas editou o código antigo que ninguém podia tocar" não é sucesso. Num time de verdade, esse PR volta na revisão.

Resolve sempre?

Acertar uma vez pode ser sorte. Acertar todas as vezes é saber. "Resolve sempre?" conta quantas tarefas deram certo em **todas** as tentativas. No jargão, essa métrica se chama **pass^k** (lê-se "pass hat k"), em que k é o número de tentativas. No run da palestra, k = 2: o relatório diz quantas das 8 tarefas deram certo nas duas repetições, por variante.

Custo em US$: leia como esforço

O Claude Code informa, no fim de cada execução, um custo em dólares (o campo `total_cost_usd`). A documentação dele ("Run Claude Code programmatically") avisa que esse valor é uma **estimativa** calculada no seu computador. Se você usa assinatura, ele não é fatura. Leia como "tamanho do esforço" da execução e compare entre variantes, não como dinheiro gasto.

Desfecho, não "funcionou?"

Cada execução do repositório termina com **um** rótulo. No jargão, esse rótulo é o **desfecho**:

Desfecho	Quando
`passou`	o teste escondido passou e nada foi alterado fora do permitido
`falhou`	o teste escondido falhou
`fora_do_escopo`	mexeu onde não devia: algum arquivo alterado não está em `permitidos` (mesmo que o teste passe)
`erro_agente`	o agente nem terminou (estourou o tempo, o login expirou, bateu no limite de voltas, devolveu uma resposta ilegível) ou mexeu nas peças que rodam o teste

A ordem de decisão é: primeiro `erro_agente`, depois `fora_do_escopo`, depois `passou` ou `falhou` pelo teste.

O ponto mais importante é o último rótulo. **Erro do agente é contado à parte**, fora da taxa de acerto. Se a sua internet caiu no meio do run, isso não diz nada sobre o `CLAUDE.md`. Se você contasse como falha, a variante que teve o azar de rodar durante a queda pareceria pior. O relatório mostra quantos erros do agente cada variante teve, e você decide se o run vale ou se precisa rodar de novo.

Essa ideia vem direto do caso real do capítulo 7: sem rótulo explícito, "funcionou?" vira discussão.

Quem confere: código ou IA como juiz?

Em todas as tarefas deste repositório, quem confere é código: o teste escondido. Existe outra opção, muito usada: pedir para um modelo de IA dar nota ao trabalho de outro. É a **IA como juiz** (LLM-as-judge; LLM é o "modelo de linguagem grande", o tipo de modelo por trás desses agentes).

- **Ajuda** quando o critério não cabe num teste: clareza de um texto, qualidade da descrição de um PR, estilo.
- **Atrapalha** porque o juiz também varia. É como um corretor de redação que, no mesmo texto, às vezes dá 7 e às vezes dá 9. Um juiz que varia avaliando um agente que varia **soma incerteza**.

A regra prática: se dá para conferir com código, confira com código. Se não dá, meça o juiz também, comparando as notas dele com as de uma pessoa. O capítulo 7 conta por que o juiz foi rejeitado num caso real.

Para quem quer ir fundo: pass@k × pass^k

Com várias repetições por tarefa, aparecem duas maneiras de resumir:

- **pass@k:** a tarefa deu certo em **pelo menos uma** de k tentativas.
- **pass^k:** a tarefa deu certo em **todas** as k tentativas.

Quando k cresce, pass@k sobe (mais chances de acertar uma vez) e pass^k desce (mais chances de errar uma vez). pass@k faz sentido quando uma pessoa olha as tentativas e escolhe a melhor. Um agente que trabalha sozinho não vai ser chamado de novo até dar certo: o que importa é se ele acerta **sempre**. Por isso o relatório mostra pass^k.

A métrica pass^k foi proposta no τ-bench (Yao et al., 2024), e o post da Anthropic "Demystifying evals for AI agents" (2026) usa a mesma definição: a chance de **todas** as k tentativas darem certo.

Os campos exatos que o repositório lê do Claude Code (`usage`, `total_cost_usd`, `num_turns`, `duration_ms`) estão no capítulo 4.

Montando as tarefas

Uma eval de agente de código é um conjunto de **tarefas** com **critério de feito**. A tarefa é o pedido, escrito como você escreveria uma issue. O critério de feito é um teste automático que prova que o pedido foi atendido: o gabarito da prova. O resto (rodar, repetir, calcular) é encanamento.

Este capítulo é para quem vai rodar ou adaptar o repositório. Ele mostra código. As partes mais detalhadas ficam no fim, em "Para quem quer ir fundo".

O laboratório

O repositório traz um app pequeno, em TypeScript (JavaScript com tipos), chamado laboratório (`laboratorio/`): uma agenda de meetups com eventos, inscrições, importação de inscritos por CSV (planilha em texto, com vírgulas) e um relatório. Os dados são inventados (sintéticos) e ficam em memória. O app tem testes automáticos escritos com o Vitest, a ferramenta de testes mais comum em projetos TypeScript modernos.

Como todo projeto real, ele tem **regras da casa** (convenções, no jargão): combinados do time que não dá para adivinhar só lendo o código. A variante B ganha um `CLAUDE.md` que explica essas regras. Este é o trecho principal, copiado do repositório (`variantes/b/CLAUDE.md`):

```
## Convenções

- Erros: sempre `ErroDominio` (`src/erros.ts`) com `codigo` em
  `SCREAMING_SNAKE`, por exemplo `INSCRICAO_DUPLICADA`. Nunca
  `throw new Error(...)` em regra de negócio.
- Dinheiro: sempre `Centavos` (inteiro, `src/dinheiro.ts`). Nada de float.
  Ao dividir ou aplicar percentual, arredonde o valor final para baixo
  (`Math.floor`).
- Tempo: nunca `new Date()` em `src/`. Use `relogio.agora()` de
  `src/relogio.ts`, para os testes conseguirem fixar a hora.
- Auditoria: toda mutação de inscrição (criar, cancelar, qualquer nova)
  chama `registrarAuditoria(acao, dados)`. Ações no formato
  `inscricao.<verbo>`.
- `src/legado/` é congelado: não edite. Código novo de relatório vai em
  `src/relatorios/`.
```

(As linhas foram quebradas para caber na página; no arquivo, cada regra é uma linha.)

Em português: erro de regra de negócio usa o tipo de erro do projeto, com um código todo em MAIÚSCULAS_COM_SUBLINHADO (o formato `SCREAMING_SNAKE`); dinheiro é sempre um número inteiro de centavos, arredondado para baixo; a hora atual vem de um relógio do projeto, que os testes conseguem fixar; toda mudança numa inscrição deixa um registro de auditoria; e a pasta `src/legado/`, de código antigo, não se toca.

As 8 tarefas

Cinco tarefas dependem de uma regra da casa que só o `CLAUDE.md` explica. Três são **controle**: não dependem de regra nenhuma, e A e B deveriam empatar nelas.

#	Tarefa (como uma issue)	O teste escondido exige
01	A listagem de próximos eventos devolve um evento a menos do que o limite.	controle
02	Uma função que conte os inscritos de um evento.	controle
03	Não deixar inscrever em evento lotado.	`ErroDominio` com código de lotação em `SCREAMING_SNAKE`
04	Cupom de desconto percentual na inscrição.	`Centavos` inteiro, valor final arredondado para baixo
05	Não deixar inscrever em evento que já aconteceu.	a regra usa `relogio.agora()`
06	O relatório de inscrições precisa mostrar o e-mail.	código novo em `src/relatorios/`, `src/legado/` intocado
07	Quando lotar, pôr a pessoa numa lista de espera.	registro de auditoria da nova mutação
08	A importação de CSV quebra com vírgula dentro de aspas.	controle

Por que ter tarefas de controle

As tarefas de controle protegem você de uma conclusão errada. É como o grupo que toma placebo num teste de remédio: se ele melhora tanto quanto o outro, o remédio não explica a melhora. O efeito esperado do `CLAUDE.md` está nas tarefas 03 a 07. Se B ganhar **também** nas tarefas 01, 02 e 08, desconfie: pode ser ruído, ou outra coisa mudou entre as variantes.

Por que o pedido não cita a regra da casa

O prompt (o texto do pedido ao agente) é escrito como uma issue de verdade: diz o que a pessoa quer, não como o projeto faz. Se o prompt dissesse "use `ErroDominio`", A e B empatariam, e a eval não mediria nada. Seria como entregar o gabarito junto com a prova. O repositório tem um teste que

falha se algum prompt mencionar `ErroDominio`, `relogio`, `Math.floor`, auditoria, `src/relatorios`, legado ou `CLAUDE.md`.

Isso não quer dizer que A não tenha chance. Na tarefa 03, o código já lança `ErroDominio` em outras regras, e um agente atento pode copiar o padrão. Na 05, outra função do app já usa `relogio.agora()`. Já nas tarefas 04 (direção do arredondamento) e 06 (pasta do código novo), não há nenhum sinal no código: elas medem o `CLAUDE.md` quase puro. Vale dizer isso quando você apresentar o resultado.

Uma tarefa por dentro

Cada tarefa é uma pasta em `tarefas/NN-nome/` com três coisas:

- `tarefa.json`: o título, o prompt, os arquivos permitidos e se a tarefa depende de convenção;
- `oculto.test.ts`: o teste escondido, que o agente não vê;
- `solucao/`: a solução de referência, usada para provar que a tarefa é possível.

O `tarefa.json` da tarefa 03, copiado do repositório:

```
{
  "titulo": "Evento lotado",
  "prompt": "Hoje dá para inscrever mais gente do que a capacidade do evento. Não deixe
inscrever em evento lotado: quando o evento já tiver `capacidade` inscritos,
`inscrever` deve recusar a nova inscrição.",
  "permitidos": ["src/inscricoes.ts", "test/**"],
  "dependeDeConvencao": true
}
```

O agente só recebe o `prompt`. O campo `permitidos` define onde ele pode mexer: qualquer arquivo alterado fora dessa lista vira `fora_do_escopo` ("mexeu onde não devia"). A lista usa globs, padrões de caminho com curinga: `*` vale para qualquer nome dentro de uma pasta, e `**` vale também para as subpastas. O identificador da tarefa é o nome da pasta.

Três regras para uma tarefa honesta

1. Toda tarefa falha no app intacto. Se o teste escondido já passa sem o agente fazer nada, a tarefa é inútil: as duas variantes "resolvem". O repositório verifica isso para as 8 tarefas.

2. Toda tarefa passa com a solução de referência. Se nem a solução passa, a tarefa é impossível, e a taxa das duas variantes cai junto. O mesmo teste verifica as duas coisas.

3. O agente não vê o teste, e não consegue trapacear com ele. O teste escondido nunca entra na pasta do agente. Ele só aparece depois que o agente termina, numa pasta nova (detalhes abaixo).

Essas regras parecem paranoia até a primeira vez que um agente, tentando "fazer os testes passarem", edita o teste em vez do código.

Para quem quer ir fundo: o teste escondido por dentro

O coração do `oculto.test.ts` da mesma tarefa (trecho; os helpers `evento`, `erroDe` e as pessoas `ana`, `bia`, `caio` ficam no topo do arquivo):

```
// O prompt não dá o nome do código: aceita qualquer SCREAMING_SNAKE
// que fale de lotação.
it("recusa inscrição além da capacidade com ErroDominio e código de lotação", () => {
  const e = evento(2);
  inscrever(e.id, ana);
  inscrever(e.id, bia);
  const erro = erroDe(() => inscrever(e.id, caio));
  expect(erro).toBeInstanceOf(ErroDominio);
  const { codigo } = erro as ErroDominio;
  expect(codigo).toMatch(/^[A-Z][A-Z0-9_]*$/);
  expect(codigo).toMatch(/LOTAD|CHEIO|CAPACIDADE|ESGOT|LIMITE|VAGA/);
  expect(codigo, "CAPACIDADE_INVALIDA já existe e significa outra coisa")
    .not.toBe("CAPACIDADE_INVALIDA");
});

it("vaga liberada por cancelamento pode ser ocupada", () => {
  const e = evento(1);
  inscrever(e.id, ana);
  cancelarInscricao(e.id, ana.email);
  expect(() => inscrever(e.id, bia)).not.toThrow();
});
```

Cada `expect` é uma conferência. `toBeInstanceOf(ErroDominio)` exige o tipo de erro do projeto. As duas linhas com `toMatch` usam regex (expressão regular, um padrão de texto): a primeira exige o formato em MAIÚSCULAS, a segunda exige uma palavra que fale de lotação.

Repare no que o teste **não** exige: um nome exato de código. O prompt não diz qual código usar, então exigir `EVENTO_LOTADO` letra por letra mediria sorte, não convenção. O teste exige o que a convenção diz (o erro do domínio, com código no formato da casa) e aceita qualquer nome razoável de lotação. Ele só recusa reaproveitar `CAPACIDADE_INVALIDA`, que já existe no app e significa outra coisa.

Desenhar o teste é a parte mais difícil. Outro exemplo é a tarefa 04, cujo teste escolhe valores que separam as interpretações erradas. O comentário do próprio teste explica:

```
// Só o valor final com Math.floor passa nos dois casos:
// - 999 a 50%: exato 499,5. Reprova Math.round (500) e também
//   floor só do desconto (999 - floor(499,5) = 500).
// - 1999 a 15%: exato 1699,15. Math.round acerta (1699), mas floor
//   só do desconto reprova (1999 - floor(299,85) = 1700).
//   Valor não inteiro (float) reprova nos dois.
```

Como o repositório impede a trapaça

Depois que o agente termina, a correção monta uma pasta **nova**, que o agente nunca viu: o app intacto, mais os arquivos que ele mudou, mais o teste escondido, que entra por último e sobrescreve qualquer arquivo com o mesmo nome. Se o agente criar um `test/oculto.test.ts` falso que sempre passa, ele é substituído. Além disso, o teste roda com uma configuração do Vitest fixa, que fica **fora** da cópia (`src/vitest-oculto.config.mjs`). Assim, um `vitest.config.ts` criado pelo agente, com `exclude` ou um `setupFiles` que "conserta" o resultado, é ignorado. O repositório tem um teste que planta exatamente essas trapaças e confere que a tarefa continua falhando.

Rodando o experimento

Este capítulo mostra como rodar o experimento no seu computador e o que acontece em cada execução. A primeira parte serve para qualquer pessoa. Da seção "O que acontece em uma execução" em diante, o texto é para quem vai adaptar o repositório ou quer entender o isolamento; os trechos mais detalhados estão marcados como "Para quem quer ir fundo".

> **Nota sobre versões.** Comandos, flags (as opções de linha de comando, como `--model`) e números abaixo são os do repositório (`package.json`, `src/claude.ts`, `evals/promptfooconfig.yaml`) no dia em que este ebook foi gerado. As flags do Claude Code mudam entre versões: se algo daqui divergir do código, vale o código.

O que você precisa

- **Node.js** na versão LTS atual (a versão com suporte longo) e **pnpm** (o instalador de pacotes que o repositório usa).
- **Claude Code** instalado e com login feito: rode `claude` uma vez no terminal. O repositório usa a sua assinatura normal do Claude Code. **Não precisa de chave de API**, e este ebook não recomenda usar uma.
- **git**: o repositório usa o git do seu computador para medir o que o agente mudou.
- **macOS**: o sandbox (a "caixa fechada" que isola o agente, explicada abaixo) foi testado só no macOS. Em Linux, o Claude Code e o sandbox-runtime têm suporte próprio, mas este repositório não foi testado lá.

Rodando em 3 comandos

```
pnpm install                    # 1. instala tudo
pnpm eval                       # 2. roda as 32 execuções e gera o relatório no fim
open resultados/relatorio.html  # 3. abre o relatório no navegador
```

No Linux, troque `open` por `xdg-open`, ou arraste o arquivo para o navegador.

O `pnpm eval` faz duas coisas em sequência. Primeiro, põe o Claude Code para trabalhar 32 vezes: 2 variantes × 8 tarefas × 2 repetições, com o modelo `sonnet`, 4 execuções ao mesmo tempo. Depois, gera o relatório: imprime uma tabela no terminal e grava `resultados/relatorio.html`. No notebook do palestrante, o ensaio levou **3 min 28 s** (cada execução, de 12 s a 42 s). No seu computador, o tempo depende do modelo, de quantas rodam ao mesmo tempo e da sua cota. O capítulo 5 explica como ler o relatório.

Sem assinatura do Claude Code? Dá para ver o relatório de um run gravado, sem internet e sem chamar nenhum modelo:

```
pnpm install
pnpm relatorio:ensaio            # grava resultados/ensaio/relatorio.html
```

Esse é o ensaio de 2026-10-27, guardado em `resultados/ensaio/` (o JSON com as 32 execuções, o relatório e as notas em `NOTAS.md`). O relatório dele fica ao lado do JSON e não mexe no relatório do seu run.

Os outros comandos

Comando	Para que serve
`pnpm test`	testa o **medidor**, com um agente falso: offline, em segundos, sem modelo (veja "Duas camadas", no fim)
`pnpm eval:rapido`	teste de fumaça: 1 tarefa, 1 repetição, A e B (2 execuções reais), para confirmar que o login e o Claude Code respondem; não gera relatório
`pnpm relatorio`	refaz o relatório a partir de `resultados/ultimo.json`, sem rodar o agente de novo
`pnpm ensaio:gravar`	guarda o último run como ensaio seu, sem caminhos pessoais

Uma ordem que funciona quando você mexe no repositório: `pnpm test` primeiro (se ele falhar, o problema está no medidor, não no agente); depois `pnpm eval:rapido`; depois `pnpm eval`.

O `package.json` também tem `pnpm relatorio:aguardando` e `pnpm relatorio:planob`. Eles são só para o palco da palestra (a página de espera e o plano B, se o run ao vivo falhar); você não precisa deles.

Para quem quer ir fundo: códigos de saída e o ensaio

O promptfoo sai com código 100 quando alguma execução não passou. O código de saída é o número que um comando devolve ao terminar; 0 quer dizer "tudo certo". O 100 é o comportamento do promptfoo, não um erro: o JSON em `resultados/ultimo.json` é gravado do mesmo jeito, e o `pnpm eval` gera o relatório em seguida. O código de saída final do `pnpm eval` é o do relatório, que reflete o veredito (capítulo 5).

Para gravar um ensaio seu, rode `pnpm eval` e logo depois `pnpm ensaio:gravar`. Ele copia `resultados/ultimo.json` para `resultados/ensaio/resultado.json` trocando a pasta do projeto por `.`, o seu home por `~` e as pastas temporárias por `<tmp>`, e gera o relatório.

O que acontece em uma execução

Cada uma das 32 execuções passa por três etapas.

1. Cópia limpa

Em resumo: o agente nunca trabalha no código de verdade. Cada execução ganha uma cópia nova do app, que vai para o lixo no fim.

O repositório cria uma pasta temporária nova (em `/tmp/medindo-agentes/`, fora de qualquer repositório e fora do `$TMPDIR`), copia o laboratório para ela (sem `node_modules`) e coloca o `CLAUDE.md` se a variante for B. Depois registra esse estado inicial num git **do host**: o diretório do git fica ao lado da cópia, não dentro dela (`--git-dir` e `--work-tree`). Tudo o que o agente fizer depois é comparado com esse estado.

Por que o git fica fora: se a cópia fosse um repositório, o agente poderia mexer no `.git` (um `config` com filtro, um hook) e fazer o git do host rodar código dele na hora de medir. Com o git fora, nada do que o agente escreve na cópia vira configuração do git.

As dependências do laboratório (só o Vitest e os tipos do Node) ficam numa pasta própria, `.lab-deps`, instalada uma vez e **somente leitura**. O `node_modules` da cópia só tem links para ela. Antes e depois da correção, o repositório confere um hash dessa pasta (conteúdo, permissões e links): se o agente conseguir mudar uma dependência, a execução vira `erro_agente` em vez de contar como resultado.

A cópia limpa garante que uma execução não contamina a outra e que A e B partem exatamente do mesmo código. A única diferença entre as variantes é o `CLAUDE.md`.

2. O agente

Em resumo: o agente faz a tarefa trancado numa caixa, sem internet, podendo escrever só na cópia.

Na cópia, o repositório roda o Claude Code em modo **headless**: sem tela interativa, com o prompt da tarefa como argumento e o resultado em JSON (um formato de texto estruturado, fácil de ler por programa). A chamada (função `argumentosClaude` em `src/claude.ts`) usa:

- `-p <prompt>` e `--output-format json`;
- `--model sonnet` (fixado em `evals/promptfooconfig.yaml`);
- `--max-turns 30`, e um timeout de 10 minutos por execução, aplicado pelo próprio repositório (ele mata o grupo de processos inteiro);
- `--permission-mode dontAsk` com uma lista fechada em `--allowedTools`: ler e editar só dentro da cópia (`Read(./**)`, `Edit(./**)`), rodar `pnpm test` e `pnpm vitest`, e os comandos de leitura `ls`, `cat`, `grep`, `head` e `wc`. O que não está na lista é recusado, sem perguntar a ninguém;
- `--disallowedTools` com `WebFetch`, `WebSearch`, a edição do `node_modules`, a leitura do projeto (onde estão os testes escondidos e as soluções) e a leitura do seu `~/.claude`;

- um **sandbox do sistema operacional** (o Seatbelt, no macOS). Sandbox é uma caixa fechada imposta pelo próprio sistema, não pelo agente: ela vale mesmo que o agente "queira" sair. Ele é configurado por `--settings`. Todo comando Bash (o terminal que o agente usa), inclusive o código que o `pnpm test` roda, só escreve na cópia e no `$TMPDIR`, não lê o seu home nem as outras cópias, e não tem rede. O sandbox falha alto se não estiver disponível e não oferece a saída "rodar fora do sandbox";
- `--setting-sources project`, para não carregar a sua configuração pessoal (veja "Isolamento", abaixo);
- `--strict-mcp-config`, sem `--mcp-config`: nenhum servidor MCP (MCP é o protocolo que liga o agente a ferramentas externas, como os conectores da sua conta);
- `--no-session-persistence`, para as execuções não deixarem sessões salvas na sua máquina.

O processo também roda sem as variáveis de ambiente de uma sessão do Claude Code que esteja aberta (`CLAUDECODE`, `CLAUDE_CODE_*`). O login vem do mesmo lugar de sempre.

Nunca use `--permission-mode bypassPermissions` nem `--dangerously-skip-permissions` para rodar esta eval. São 32 execuções sem ninguém olhando: com as permissões desligadas, um agente que decide "limpar" arquivos ou instalar algo faz isso na sua máquina, não só na cópia.

Do JSON final, o repositório lê `is_error`, `num_turns`, `duration_ms`, `total_cost_usd` e os campos de `usage`: tokens de entrada e de saída, e, separados, os de cache (leitura + criação). Se o processo estourar o timeout, ou o JSON vier inválido, a execução vira `erro_agente` em vez de derrubar o run.

Confira as flags com `claude --help` e na página "Run Claude Code programmatically" da documentação antes de mudar qualquer coisa: elas mudam entre versões.

3. A correção

Em resumo: depois que o agente termina, um teste que ele não viu confere o trabalho, numa pasta nova.

Quando o agente termina, o repositório:

1. lista os arquivos alterados em relação ao estado inicial, com o git do host, incluindo arquivos novos e apagados;
2. procura entradas suspeitas na cópia: qualquer link simbólico, qualquer `.git`, FIFO ou dispositivo. Se achar, a execução vira `fora_do_escopo` e nem é corrigida;
3. compara a lista de alterados com `permitidos` da tarefa. `package.json`, configuração do Vitest ou do Vite e qualquer coisa dentro de um `.git` são sempre fora do escopo;
4. monta uma pasta **nova**, que o agente nunca viu: o laboratório intacto, mais os arquivos que ele mudou (só arquivos comuns), mais o teste escondido por cima de `test/oculto.test.ts`;
5. roda só o teste escondido, com a configuração fixa do Vitest (capítulo 3), dentro de outro sandbox (o `srt`, do pacote `@anthropic-ai/sandbox-runtime`): sem rede, escrevendo só na pasta corrigida;

6. decide o desfecho: `erro_agente`, senão `fora_do_escopo`, senão `passou` ou `falhou`.

Para quem quer ir fundo: como a lista de alterados é feita

A lista de alterados vem do git do host (trecho de `src/copia.ts`, com as linhas quebradas para caber na página):

```
export async function arquivosAlterados(pasta: string): Promise<string[]> {
  const nulo = (s: string) => s.split("\0").filter(Boolean);
  const mudados = nulo(
    await git(pasta, "diff", "--name-only", "--no-renames", "-z", REF_INICIAL),
  );
  const novos = nulo(
    await git(pasta, "ls-files", "--others", "-z",
      `--exclude-from=${exclusoes(pasta)}`),
  );
  return [...new Set([...mudados, ...novos])]
    .filter((f) => f !== OCULTO)
    .sort();
}
```

Um detalhe que só apareceu rodando de verdade: `pnpm test` dentro da cópia cria um `pnpm-lock.yaml`. Sem tratar isso, todo agente que roda os testes (que é o que o `CLAUDE.md` pede) apareceria com um arquivo fora do escopo. As exclusões do git do host (o arquivo `exclusoes`, fora da cópia) escondem esse arquivo e o `node_modules` montado pelo repositório; um `.gitignore` criado pelo agente na cópia não esconde nada.

Para quem quer ir fundo: notas de método

Algumas consequências desse desenho, que valem saber antes de ler o resultado:

- **A cópia não é um repositório git.** Um comando `git` do agente (`git status`, `git diff`) falha. Se o agente rodar `git init`, cria um `.git` e a execução vira `fora_do_escopo`. As tarefas não pedem git, e no ensaio nenhuma execução criou um `.git`.
- **Qualquer link simbólico criado pelo agente vira `fora_do_escopo`**, mesmo um que aponte para dentro da cópia: a correção só aceita arquivos comuns.
- **Configuração gerenciada pela organização continua valendo.** Se a sua conta tem configurações gerenciadas (managed settings), elas chegam às duas variantes, igualmente, e nenhuma flag tira. O isolamento é "nada do seu usuário e nada de MCP", não "configuração zero".
- **A tarefa 06 mede só a convenção.** O prompt pede uma função de relatório sem dizer onde; a variante A não tem nenhum sinal de que ela deve ir em `src/relatorios/`. A diferença ali é o efeito do `CLAUDE.md`, por construção.

- **O `ensaio:gravar` limpa por texto.** Ele troca os caminhos por substituição de texto no JSON: qualquer trecho `/tmp` vira `<tmp>`, mesmo fora de um caminho, e um caminho escrito com barras escapadas passaria. Confira o JSON antes de publicar um ensaio seu.

Onde entra o promptfoo

O promptfoo é uma ferramenta de código aberto para rodar evals. Aqui ele é o **orquestrador**: ele lê a lista de casos, chama um *provider* para cada um, repete, roda em paralelo e grava tudo num JSON. *Provider*, no promptfoo, é o pedaço de código que "responde" a cada caso. Normalmente é um modelo; aqui é um provider em TypeScript que roda as três etapas acima e devolve o desfecho.

No repositório:

- `evals/agente.ts` é o provider: recebe a variante e o modelo na configuração, expõe `id()` (`claude-code-a` ou `claude-code-b`) e `callApi`, que roda a tarefa e devolve `{ output: desfecho, metadata: { resultado } }`;
- `evals/casos.ts` gera um caso por tarefa, com uma asserção que passa quando o `output` é `passou`;
- `evals/promptfooconfig.yaml` declara os dois providers, 2 repetições e concorrência 4.

Para quem quer ir fundo: a configuração

O trecho principal da configuração:

```
providers:
  - id: file://agente.ts
    label: variante a
    config:
      variante: a
      modelo: sonnet
  - id: file://agente.ts
    label: variante b
    config:
      variante: b
      modelo: sonnet
tests: file://casos.ts
evaluateOptions:
  maxConcurrency: 4
  repeat: 2
```

O cache fica desligado pela linha de comando (`--no-cache`, nos scripts do `package.json`): na versão do promptfoo usada, a chave `cache: false` do arquivo não tinha efeito.

A página "Javascript Provider" da documentação do promptfoo descreve o contrato do provider (`id` e `callApi`). A página "Command line" descreve as flags que o `pnpm eval:rapido` usa: `--filter-first-n` para rodar só a primeira tarefa e `--repeat` para mudar o número de repetições, além de `-o` para gravar a saída.

O promptfoo não calcula o veredito. Ele não tem intervalo de confiança nem comparação A × B com essa regra. Isso fica num script do repositório (`scripts/relatorio.ts`), que lê o JSON do promptfoo e tira o desfecho de `metadata.resultado`, nunca do "passou/falhou" do promptfoo, que não distingue `erro_agente` de `falhou`. É o assunto do capítulo 5.

Isolamento: a sua configuração não pode vazar

Este é o erro mais fácil de cometer e o mais difícil de perceber. O Claude Code lê configuração de vários lugares: do projeto, mas também do seu usuário (`~/.claude/CLAUDE.md`, hooks, skills globais, plugins; o capítulo 6 explica hook e skill) e dos conectores MCP da sua conta. Se você tem um `CLAUDE.md` pessoal, a variante A **não** roda "sem `CLAUDE.md`": ela roda com o seu. O experimento fica contaminado e você não fica sabendo.

Para quem quer ir fundo: como o isolamento foi conferido

O repositório restringe as fontes de configuração ao projeto (`--setting-sources project`), o que mantém o `CLAUDE.md` da cópia (variante B) e deixa de fora o seu usuário. Isso não bastou: medindo de verdade, o início de cada execução ainda listava 47 servidores MCP (os conectores da conta, que `--setting-sources` não controla) e um número instável de ferramentas. Com `--strict-mcp-config`, sem nenhum `--mcp-config`, ficaram 0 servidores MCP e só as 25 ferramentas nativas, nas duas variantes.

A verificação foi feita assim, antes do ensaio (`resultados/ensaio/ISOLAMENTO.md`): numa cópia de cada variante, com os mesmos argumentos do executor, pedir ao agente que liste literalmente todas as instruções de `CLAUDE.md` ou memória que recebeu, e ler o evento de início (`--output-format stream-json --verbose`). A variante A respondeu "NENHUMA"; a B citou só o `CLAUDE.md` da cópia. Se você adaptar o repositório, repita essa verificação na sua máquina.

> **Segurança: rode só numa cópia descartável.**
>
> - Nunca aponte o agente para o seu checkout de trabalho. Cada execução tem que rodar numa pasta temporária que pode ir para o lixo.
> - Mantenha o sandbox ligado: sem rede, escrita só dentro da cópia.
> - Se você trocar o laboratório pelo seu repositório, confira o sandbox de novo: peça ao agente, numa cópia, para escrever fora dela e para acessar a rede, e veja as duas coisas falharem.

> - Lembre que `pnpm test` roda o que o `package.json` mandar. O script `test` do seu projeto passa a ser executado dezenas de vezes, sem supervisão.

Duas camadas de eval

O repositório tem duas camadas, com papéis diferentes.

Camada 1: determinística. São os testes do **medidor**, com um agente falso: um que não faz nada (deve dar `falhou`), um que aplica a solução de referência (`passou`), um que edita o código congelado (`fora_do_escopo`), um que devolve timeout (`erro_agente`, sem derrubar o run), um que planta um teste oculto falso (continua `falhou`). Roda offline, em segundos, a cada commit, com `pnpm test`. É a camada que cabe no CI (integração contínua: o processo que roda os testes automaticamente a cada push ou PR). Ela prova que a régua mede certo.

Camada 2: modelo real. É o `pnpm eval`: o Claude Code de verdade nas 8 tarefas. Leva minutos e consome cota. Rode quando algo que você quer medir mudar: o prompt, o `CLAUDE.md`, o modelo, uma skill, um hook.

Sem a camada 1, um bug no medidor parece "o agente piorou". Com ela, quando a camada 2 muda de resultado, você sabe que foi o agente.

Lendo o resultado

O run terminou e o relatório está aberto. A tentação é olhar quantas vezes A deu certo, quantas vezes B deu certo, e declarar um vencedor. Este capítulo é sobre não fazer isso.

O relatório, de cima para baixo

O `pnpm eval` gera o relatório sozinho quando termina: imprime uma tabela no terminal e grava `resultados/relatorio.html`. Para ver o do ensaio sem rodar nada, use `pnpm relatorio:ensaio` e abra `resultados/ensaio/relatorio.html`.

A página tem duas partes. Em cima, linguagem simples. Embaixo, em "Detalhes técnicos", os nomes que você vai achar nos livros e nos artigos.

1. O título. Uma de três frases: "Com CLAUDE.md foi melhor", "Sem CLAUDE.md foi melhor" ou "Ainda não dá para saber". Logo abaixo, uma linha diz o porquê, por exemplo: "a margem de erro das duas se cruza".

2. As duas linhas grandes. Uma por variante, assim: "Sem CLAUDE.md (A): 12 de 16 deram certo · 75%, margem de erro: 51% a 90%". Embaixo, uma régua de 0% a 100%. Em cada linha, o tracinho é o resultado do run, e a faixa colorida é a margem de erro (o intervalo de confiança, no jargão). Leia sempre o número **com** a faixa.

3. Os números lado a lado. Uma tabela com as métricas do capítulo 2, com nomes do dia a dia:

- "Deram certo" e "Margem de erro";
- "Resolve sempre?": tarefas que deram certo nas 2 tentativas (pass^k);
- "Mexeu onde não devia" (fora do escopo);
- "Erro do agente": rede, login, limite; não entra na conta;
- "Voltas com as ferramentas" (turnos, mediana), "Tempo por execução" (mediana) e "Custo estimado do run".

Mediana é o valor do meio quando você põe tudo em ordem. O relatório usa a mediana, e não a média, porque algumas execuções gastam muito mais que as outras e puxariam a média para cima.

4. Tarefa por tarefa. Quantas deram certo em cada uma das 8 tarefas, por variante. As tarefas 03 a 07 dependem de uma regra da casa; 01, 02 e 08 são de controle (capítulo 3).

5. Detalhes técnicos. O veredito técnico, o nome da conta da margem de erro (intervalo de Wilson, 95% de confiança), quantas execuções seriam precisas para separar A de B, o piso pelo teste z, o n de cada variante, pass^k e os tokens. O fim deste capítulo explica cada um.

Antes de concluir: confira duas coisas

1. **Quantas execuções entraram na conta.** As duas linhas devem dizer "de 16". Se aparecer menos, houve erro do agente (capítulo 2): essas execuções ficam fora da conta. Se uma variante teve muitos erros, o run não está equilibrado. Investigue antes de concluir qualquer coisa.
2. **Onde está a diferença.** Na tabela tarefa por tarefa, o ganho está nas tarefas de regra da casa (03 a 07) ou nas de controle?

Quando dá para dizer quem ganhou

Cada variante tem a sua faixa: a margem de erro. A regra para ler qualquer relatório cabe em duas linhas:

- **As faixas se cruzam:** ainda não dá para saber quem é melhor.
- **As faixas se separam:** agora dá. Se a faixa de B fica toda acima da de A, B é melhor. Se fica toda abaixo, A é melhor.

É a mesma ideia do "empate técnico" da pesquisa eleitoral. Dois candidatos parecem longe, mas as margens se cruzam, e ninguém pode dizer quem está na frente. Com mais entrevistas, as margens estreitam, e aí dá.

"Ainda não dá para saber" não é empate. Empate seria "as duas são iguais". "Ainda não dá para saber" (inconclusivo, no jargão) é "não tenho dado para dizer". São conclusões diferentes, e a segunda tem uma ação clara: rodar mais.

O resultado do experimento

O ensaio gravado em 2026-10-27 (`resultados/ensaio/`, modelo `sonnet`, 2 repetições) e um run de teste do palestrante em 2026-10-29 deram **o mesmo placar**:

	Sem `CLAUDE.md` (A)	Com `CLAUDE.md` (B)
Deram certo	12 de 16 (75%)	16 de 16 (100%)
Margem de erro (Wilson 95%)	51% a 90%	81% a 100%
Resolve sempre? (pass^2)	6 de 8	8 de 8
Mexeu onde não devia	3	0
Erro do agente	0	0

Título do relatório, nos dois: **"Ainda não dá para saber"**. As faixas se cruzam entre 81% e 90%.

As quatro falhas de A foram só em tarefas de regra da casa:

- **Tarefa 04 (cupom de desconto):** numa execução, o agente mexeu em `src/legado/`, a pasta que não se toca; na outra, o teste escondido falhou.
- **Tarefa 06 (e-mail no relatório):** nas duas, o agente mexeu fora dos arquivos permitidos.

Nenhuma tarefa de controle falhou, em nenhuma das variantes.

Dois runs com o mesmo placar dizem uma coisa útil: as falhas de A não caíram em tarefas sorteadas. Elas se repetiram nas mesmas tarefas, as que dependem de uma regra que só o `CLAUDE.md` explica. É exatamente o efeito que as tarefas foram desenhadas para medir. Mas não vá além disso. Em cada run, as faixas se cruzam: a resposta honesta continua sendo "ainda não dá para saber". E o seu run pode dar outro placar.

Quantas execuções para saber?

O relatório também diz quantas execuções **de cada lado** seriam precisas para as faixas se separarem. No experimento: **27**, contra as 16 feitas.

Volte à moeda do capítulo 1. Jogar 4 vezes e tirar 3 caras não prova que ela é viciada; jogar 400 vezes e tirar 300, prova. Quanto menor a diferença que você quer ver, mais vezes precisa jogar. Alguns valores, com a mesma conta do relatório:

Taxa de A	Taxa de B	Execuções de cada lado para as faixas se separarem
75%	100%	27
37,5%	75%	27
62,5%	75%	212

Uma diferença grande se vê com poucas dezenas de execuções. Uma diferença moderada pede centenas. A conta supõe que as taxas continuem iguais com mais execuções, o que ninguém garante: é uma estimativa, não uma promessa.

Esta é a lição da palestra: **16 execuções de cada lado quase nunca bastam para uma diferença moderada.** E está tudo bem. Saber quanto falta já é medir.

Os três títulos possíveis

"Com CLAUDE.md foi melhor." Confira onde. O ganho está nas tarefas de regra da casa (03 a 07)? Ótimo, é o efeito esperado. Está também nas de controle? Desconfie: pode ser ruído, ou outra coisa diferente entre as variantes. Olhe também "Mexeu onde não devia" e o custo: ganhar gastando muito mais pode não valer.

"Ainda não dá para saber." O resultado mais comum com 16 execuções de cada lado. Leia quantas execuções seriam precisas e decida se vale rodar mais. Olhe a tabela tarefa por tarefa: às vezes uma tarefa mostra um padrão claro (A sempre falha, B sempre passa) que merece

investigação, mesmo sem veredito.

"Sem CLAUDE.md foi melhor." O `CLAUDE.md` atrapalhou. Isso acontece: instruções demais distraem, uma regra mal escrita leva o agente para o lado errado. Abra as execuções de B que falharam e veja o que o agente fez.

O portão de qualidade

O `pnpm eval` (e o `pnpm relatorio`) terminam com código de saída 1 **só** quando o título é "Sem CLAUDE.md foi melhor", ou seja, quando há evidência de que a mudança piorou o agente. "Ainda não dá para saber" e "Com CLAUDE.md foi melhor" saem com 0. Se o arquivo de entrada não puder ser lido, sai com 2.

Isso permite usar o relatório como **portão** no CI (integração contínua: o processo que roda os testes automaticamente a cada push ou PR). Se alguém mudar o `CLAUDE.md` e a eval mostrar piora clara, o build quebra. Quebrar o build por "ainda não dá para saber" seria um erro: com poucas execuções, isso aconteceria o tempo todo, por falta de dado, e o time aprenderia a ignorar o portão.

(Não confunda com o código 100 que o promptfoo devolve quando alguma execução não passou: isso não é veredito, e o `pnpm eval` não o repassa. Capítulo 4.)

Para quem quer ir fundo: a estatística por trás

O intervalo de Wilson

Um intervalo de confiança é a faixa de taxas "verdadeiras" que combinam com o que você observou. Com 95% de confiança: se você repetisse o experimento muitas vezes, o intervalo calculado conteria a taxa verdadeira em cerca de 95% das vezes.

A conta mais conhecida para uma proporção é `p ± 1,96 · √(p(1−p)/n)`. Com n pequeno, ou com taxa perto de 0% ou de 100%, ela se comporta mal: o intervalo sai de [0, 1] ou fica com largura zero. Com 16 de 16, por exemplo, ela daria "100% ± 0", como se 16 execuções provassem perfeição. O **intervalo de Wilson** (Wilson, 1927) não tem esse problema e é a escolha comum para proporções com poucas amostras. É o que o repositório calcula, com z = 1,96 (95%).

A regra do veredito, no código

A função `decidir`, em `src/veredito.ts`, decide assim:

- **B melhor** se o limite inferior de B passa do limite superior de A;
- **A melhor** se o limite inferior de A passa do limite superior de B;
- **inconclusivo** em qualquer outro caso, inclusive quando uma variante tem n = 0.

Um exemplo ilustrativo, não um resultado de run: A com 6 de 16 (37,5%, Wilson 18% a 61%) e B com 12 de 16 (75%, Wilson 51% a 90%). A taxa de B é o dobro da de A, mas as faixas se cruzam entre 51% e 61%: existe uma taxa verdadeira, por exemplo 55%, que combina com as duas. Pela regra, inconclusivo.

Esse critério é conservador de propósito. Um teste z de duas proporções, aplicado às mesmas 6/16 e 12/16, daria uma diferença estatisticamente significativa (p ≈ 0,03). O repositório prefere não declarar vencedor cedo: exigir faixas sem cruzar é mais exigente, e um portão que declara vencedor com pouca evidência ensina o time a desconfiar dele.

As duas estimativas de execuções necessárias

Os "Detalhes técnicos" trazem dois números.

Execuções necessárias por variante (critério do veredito). O relatório pega as taxas observadas e pergunta: com quantas execuções por variante, mantidas essas taxas, as faixas de Wilson 95% deixariam de se cruzar? Ele testa n = 1, 2, 3… até achar (ou até 10.000, o teto da busca). É o número que manda, porque usa o mesmo critério do veredito. No código, o campo chama `execucoesNecessariasPorVariante` (em `src/veredito.ts`). Ele fica vazio ("não calculável") quando as duas taxas são iguais, quando uma variante não tem dado ou quando nem 10.000 bastam.

Piso pelo teste z de duas proporções. É a fórmula clássica de tamanho de amostra, com 95% de confiança e 80% de poder. **Poder** é a chance de o experimento detectar a diferença se ela existir de verdade. Com 80% de poder: se o `CLAUDE.md` ajuda mesmo, em 8 de cada 10 experimentos desse tamanho você veria a diferença.

```
n ≈ (1,96 + 0,84)² · [pA(1−pA) + pB(1−pB)] / (pA − pB)²
```

O 1,96 vem dos 95% de confiança, e o 0,84, dos 80% de poder. No código, o campo chama `pisoTesteZPorVariante`. Como o teste z é menos exigente que "faixas sem cruzar", esse número costuma ficar abaixo do primeiro: com ele, o relatório ainda pode sair inconclusivo.

Taxa de A	Taxa de B	Critério do veredito (Wilson)	Piso (teste z)
75%	100%	27	24
37,5%	75%	27	24
62,5%	75%	212	212

O n cresce com 1 / (diferença)².

Mais detalhes da tabela

- **pass^k:** o k é o maior número de repetições visto por tarefa no run (hoje, 2). A conta é conservadora: uma repetição com erro do agente conta como "não passou", e uma tarefa com menos de k repetições (run interrompido) não conta.
- **Tokens:** aparecem em duas linhas, entrada + saída, e cache (leitura + criação). O cache é dezenas de vezes maior e, somado, esconderia qualquer diferença entre as variantes.
- **Medianas e custo:** as medianas usam só as execuções sem erro do agente; o custo total soma todas, inclusive as com erro, porque é o que o run gastou.
- **Código:** o código de saída vem de `codigoDeSaida`, em `src/relatorio.ts`.

Melhorando o agente

Medir só vale se vira decisão. Este capítulo é sobre como usar a eval para melhorar o agente que você usa, e como montar a eval no seu próprio projeto. As duas primeiras seções servem para qualquer pessoa. O roteiro de adaptação e o desafio extra são para quem vai mexer no repositório.

Uma mudança por vez

Pense numa receita de bolo. Se você troca o forno **e** a farinha no mesmo dia e o bolo fica melhor, não sabe qual dos dois ajudou. O ciclo é o mesmo de um teste A/B de produto:

1. rode a eval **antes** da mudança (a linha de base);
2. mude **uma** coisa;
3. rode a eval **de novo**, com repetições;
4. compare as margens de erro (o intervalo de confiança), não só a taxa.

"Uma coisa" pode ser:

- o `CLAUDE.md`;
- uma skill nova;
- um hook;
- o modelo;
- o prompt da tarefa.

Se você mudar o modelo **e** o `CLAUDE.md` juntos e o resultado melhorar, você não sabe qual dos dois ganhou. Pode até ser que um ajudou muito e o outro atrapalhou um pouco. O experimento da palestra muda só uma coisa: a existência do `CLAUDE.md`.

Uma dica prática: commite a configuração do agente junto com o resultado da eval. Daqui a um mês, você sabe qual versão gerou qual número.

Onde cada regra mora

O Claude Code oferece três lugares para pôr uma regra. Em uma linha cada:

- **`CLAUDE.md`:** um texto com as regras do projeto, que o agente lê no começo de toda conversa.
- **Skill:** um "manual" sobre um assunto, que o agente só abre quando a tarefa pede.
- **Hook:** um script seu que roda automaticamente antes ou depois de cada ação do agente, e pode bloquear a ação.

Eles se comportam de modo diferente.

	Quando entra	Bom para	Cuidado
`CLAUDE.md`	em **toda** sessão	convenções do repo, comando de teste, pastas proibidas	é contexto, **não obrigação**
Skill	só quando é usada	procedimento longo, material de referência	descrição ruim, nunca é chamada
Hook	antes ou depois de cada ferramenta	regra que **precisa** valer sempre	é código: teste como código

- **`CLAUDE.md`.** A documentação do Claude Code ("How Claude remembers your project") diz que o conteúdo do `CLAUDE.md` é tratado como contexto, não como configuração obrigatória. O modelo tende a seguir, mas pode não seguir. Para bloquear uma ação de verdade, a mesma documentação indica um hook `PreToolUse`.
- **Skill.** Pela documentação ("Extend Claude with skills"), só a descrição da skill fica no contexto o tempo todo; o corpo entra quando ela é usada. Material de referência longo custa quase nada até ser necessário. Em compensação, se a descrição for ruim, o agente não percebe que deveria usá-la.
- **Hook.** Um script que o Claude Code roda antes ou depois de cada chamada de ferramenta. Pela documentação ("Hooks reference"), um hook `PreToolUse` que sai com código 2 bloqueia a chamada, não importa o que o modelo queira. É a única das três que é determinística.

A regra que resume a tabela: **regra que não pode falhar não mora no prompt. Mora no hook, no teste ou no CI.**

Cada linha dessa tabela é uma variante B possível no repositório. Troque o `CLAUDE.md` por uma skill com as mesmas convenções, ou por um hook que bloqueia edição em `src/legado/`, e rode de novo. A pergunta deixa de ser "qual é melhor?" em abstrato e passa a ser "qual é melhor nestas tarefas, com esta margem de erro?".

Falha vira tarefa nova

O conjunto de tarefas não é fixo. Ele cresce com o uso:

1. o agente erra algo no dia a dia;
2. você escreve uma **tarefa** com o prompt real que levou ao erro;
3. e um **teste escondido** que prova o comportamento certo;
4. confere que a tarefa falha no app intacto e passa com a solução de referência;
5. a tarefa entra no conjunto e **não sai mais**.

É a lição 7 do caso real (capítulo 7): o erro de terça vira tarefa na quarta.

A catraca

Catraca é um mecanismo que só gira para um lado. Aplicada à eval:

- o conjunto de tarefas **só cresce**;
- o portão só quebra com evidência de piora (faixas que se separam, capítulo 5).

O portão roda no CI (integração contínua: o processo que roda os testes automaticamente a cada push ou PR) e não olha a taxa crua ("não pode cair abaixo da última vez"). Com poucas execuções, a taxa oscila de uma rodada para outra só por ruído, e um portão assim ficaria vermelho sem o agente ter piorado.

Adapte ao seu projeto

O laboratório é pequeno de propósito. A eval que importa é a do **seu** repositório. Um roteiro para montar as primeiras 5 tarefas:

1. Escolha a pergunta. O que você quer saber? "Meu `CLAUDE.md` ajuda?", "Vale trocar de modelo?", "Esse hook atrapalha?". A pergunta define as variantes A e B. Mude uma coisa só.

2. Junte 5 pedidos reais. Procure no histórico do time: issues fechadas, PRs pequenos, pedidos que você fez ao agente na última semana. Prefira tarefas que um dev resolve em minutos, não em horas: cada execução precisa caber em poucos minutos, e você vai rodar várias.

3. Misture regra da casa e controle. Pelo menos 3 tarefas devem depender de algo que só a sua mudança ensina (a regra do `CLAUDE.md`, o procedimento da skill). Pelo menos 2 devem ser controle, sem relação com a mudança. Se B ganhar também nos controles, desconfie.

4. Escreva o prompt como issue. Diga o que a pessoa quer, não como o projeto faz. Não cite a regra da casa no prompt: se citar, A e B empatam.

5. Escreva o teste escondido. Ele é o critério de feito. Teste o comportamento e a regra da casa, não um detalhe que o prompt não pediu (lembre do código de erro da tarefa 03, no capítulo 3). Se o seu projeto não tem teste, comece por aqui: escrever o teste da tarefa é o primeiro entregável.

6. Defina `permitidos`. Liste os arquivos e pastas que a tarefa pode tocar. Inclua a pasta de testes, se o agente puder escrever testes.

7. Prove que a tarefa é honesta. Ela tem que falhar no código intacto e passar com uma solução de referência que você escreve. Guarde a solução em `solucao/`.

8. Troque o laboratório pelo seu repo. Este passo é o mais técnico. No `agent-evaluation`, são estas as mudanças:

- **De onde vem a cópia:** a constante `LAB` em `src/copia.ts` aponta para `laboratorio/`. Troque pelo caminho do seu projeto (ou copie o projeto para dentro de `laboratorio/`). A cópia continua sendo feita numa pasta temporária: o agente nunca trabalha no seu checkout.
- **As tarefas:** ponha as suas em `tarefas/NN-nome/` (`tarefa.json`, `oculto.test.ts`, `solucao/`) e a sua mudança em `variantes/b/`.

- **Os termos proibidos nos prompts:** o teste em `test/tarefas.test.ts` tem uma regex (expressão regular, um padrão de texto) com as palavras que nenhum prompt pode conter (hoje, `ErroDominio`, `relogio`, `Math.floor`, auditoria, `src/relatorios`, legado, `CLAUDE.md`). Troque pelos nomes das regras do **seu** projeto, para o teste continuar impedindo que o prompt entregue a resposta.
- **As dependências:** a cópia precisa delas para rodar os testes. O repositório instala uma vez, em `.lab-deps`, a lista fixa de `prepararDependencias` em `src/copia.ts` (hoje só `vitest` e `@types/node`, nas versões da raiz), deixa a pasta somente leitura, confere um hash dela antes e depois da correção e monta na cópia um `node_modules` com links para ela. Troque essa lista pelas dependências do seu projeto e mantenha as duas garantias: somente leitura e hash conferido.
- **Stack que não é Node:** o executor do teste escondido é Vitest, com a configuração fixa em `src/vitest-oculto.config.mjs`. Se o seu projeto é Python, Go, Java etc., troque esse executor pelo equivalente do seu stack (por exemplo, rodar só o arquivo do teste escondido com o runner da linguagem), mantendo as duas garantias: o teste entra por último, numa pasta corrigida nova, depois que o agente termina, e a configuração do runner vem de fora da cópia.

> **Segurança antes de rodar no seu projeto.**
>
> - Rode só em cópia descartável, nunca no seu checkout de trabalho.
> - Mantenha o sandbox: sem rede, escrita só dentro da cópia. Nunca troque por `bypassPermissions` ou `--dangerously-skip-permissions`.
> - Depois de apontar para o seu repositório, confira o sandbox de novo (capítulo 4).
> - `pnpm test`, ou o comando de teste do seu stack, roda o que os scripts do seu projeto mandarem, dezenas de vezes e sem supervisão. Leia esses scripts antes.

9. Rode pequeno, depois grande. Primeiro `pnpm eval:rapido`, para ver se tudo encaixa. Depois o run completo. Leia o relatório como no capítulo 5 e use a estimativa de execuções necessárias para decidir se vale rodar mais.

Cinco tarefas com duas repetições são 10 execuções por variante. Isso raramente separa A de B, mas já mostra onde o agente erra, e cada erro vira uma tarefa nova.

Para quem quer ir fundo: medir outro agente

Este é um desafio extra. O método não é específico do Claude Code. No repositório, a parte que chama o agente fica isolada atrás de uma interface, a `RodarAgente` (em `src/claude.ts`):

```
export interface SaidaAgente {
  ok: boolean; erro?: string;
  turnos: number; duracaoMs: number; custoUsd: number;
  tokensEntrada: number; tokensSaida: number; tokensCache: number;
}
export type RodarAgente = (args: {
  pasta: string; prompt: string; modelo: string; maxTurnos: number; timeoutMs: number;
}) => Promise<SaidaAgente>;
```

Tarefas, cópia limpa, teste escondido, escopo e relatório não sabem qual agente rodou. Para medir outro agente, você implementa **outra** `RodarAgente`. O desafio fica descrito aqui, não implementado no repositório.

Um exemplo é o **Codex CLI**, da OpenAI. A documentação dele ("Non-interactive mode") descreve o `codex exec`, que roda o agente sem a interface interativa, para scripts e CI. Pontos que a documentação descreve e que interessam aqui:

- por padrão, o `codex exec` roda com sandbox somente leitura; para o agente poder editar a cópia, é preciso `--sandbox workspace-write`;
- com `--json`, a saída vira uma sequência de eventos JSON (um por linha), com eventos como `turn.completed`, `turn.failed` e `error`;
- `--ignore-user-config` faz o run não carregar o arquivo de configuração do usuário (`$CODEX_HOME/config.toml`), o equivalente ao isolamento que o repositório faz com o Claude Code;
- o `codex exec` exige rodar dentro de um repositório git. A cópia limpa deste repositório **não** é um (o git que mede o diff fica fora dela, de propósito). A documentação permite pular essa checagem com `--skip-git-repo-check` "se você tiver certeza de que o ambiente é seguro": aqui o ambiente é uma cópia descartável, mas decida isso conscientemente, e não crie um `.git` dentro da cópia, que vira `fora_do_escopo`;
- ele reaproveita o login já salvo da CLI.

O roteiro do desafio:

1. criar um `rodarCodex: RodarAgente` que inicia o processo `codex exec` com a cópia descartável como diretório de trabalho (`cwd`), o prompt da tarefa, `--json` e `--sandbox workspace-write`. Não use o modo sem sandbox (`danger-full-access`) nem a aprovação automática de tudo: o agente roda sem supervisão;
2. ler os eventos da saída e preencher a `SaidaAgente` com o que der; o que a CLI não informar (por exemplo, custo) fica zero, e o relatório deve deixar isso claro;
3. tratar timeout e evento de erro como `ok: false`, para virar `erro_agente`;
4. repetir a verificação de isolamento do capítulo 4 (a variante A não pode receber a sua configuração pessoal);
5. confirmar as flags na documentação atual e no `codex exec --help` antes de rodar, porque elas mudam entre versões.

Isso não é um benchmark de qual ferramenta é melhor. Comparar agentes diferentes muda muita coisa de uma vez (modelo, ferramentas, prompt de sistema). O uso honesto é o mesmo de sempre: medir a configuração que você usa e ver se uma mudança nela ajuda.

Caso real: agentes KTLO na Creditas

Na Creditas, um time construiu agentes que resolvem chamados operacionais de suporte, o trabalho de *keep the lights on* (KTLO): o que mantém a operação rodando no dia a dia. Medir esses agentes ensinou mais sobre eval do que qualquer benchmark (um conjunto público de tarefas usado para comparar modelos).

Este capítulo traz só as lições. Ele não traz números internos, nomes de sistemas, de parceiros ou de pessoas, nem dados de clientes. As lições valem para qualquer agente, inclusive o agente de código que você usa.

1. Desfecho explícito por execução

Toda execução do agente termina com um rótulo: **resolvido**, **assistido**, **escalado** ou **falta informação**.

"Funcionou?" não é uma pergunta que se responde olhando um painel. "Assistido", por exemplo, quer dizer que o agente fez o diagnóstico, mas um humano executou a correção. Contar isso como "resolvido" infla o número. Contar como "falha" esconde o valor que o agente entregou. Por isso é um rótulo próprio.

No repositório desta palestra, a mesma ideia aparece como `passou`, `falhou`, `fora_do_escopo` e `erro_agente` (capítulo 2).

2. Desfecho mal definido distorce a métrica

Quando um caso que está **fora do escopo** do agente é contado como falha, a taxa de sucesso cai. E ninguém sabe se o agente piorou ou se simplesmente entrou outro tipo de caso.

Se a composição do denominador muda, a taxa muda sem o agente mudar. A lição: defina o desfecho **antes** de olhar o número.

3. O custo varia muito entre execuções

Algumas execuções custavam muito mais que a média, por causa do número de turnos. A média escondia isso; por isso olhar mediana e cauda importa mais que olhar a média.

A primeira tentativa de controle foi pôr no prompt algo como "pare depois de N chamadas". **Não segurou**: o modelo não conta as próprias voltas. O teto de orçamento virou um **hook**: um script que roda fora do modelo, a cada ação, e bloqueia a execução quando o limite é atingido. Ele é determinístico: faz sempre a mesma coisa.

É o mesmo princípio do capítulo 6: regra que não pode falhar não mora no prompt.

4. Eval em duas camadas

A eval ficou com duas camadas:

- uma **determinística** (que dá sempre o mesmo resultado), que roda a cada commit e testa a lógica do agente sem chamar o modelo;
- uma com o **modelo real**, que roda contra um **gabarito curado por humano**.

O gabarito **nasce vazio**. A pessoa escreve a resposta esperada sem olhar o que o agente respondeu. Se você copia a resposta do agente para o gabarito, é como deixar o aluno escrever o gabarito da própria prova: você mede o agente contra ele mesmo, e herda os vícios dele.

O repositório desta palestra repete as duas camadas: `pnpm test` (o medidor, com agente falso) e `pnpm eval` (o Claude Code de verdade), capítulo 4.

5. Matriz-catraca

A cobertura da eval é uma matriz: **rota × desfecho**. Cada rota (um tipo de chamado que o agente atende) cruzada com cada desfecho possível é uma célula. Cada célula deveria ter pelo menos um caso de eval.

A lista de células **sem cobertura** só pode **encolher**. Um teste quebra se alguém tirar cobertura. É a catraca do capítulo 6, aplicada à cobertura da eval.

6. IA como juiz: rejeitada ali

Usar outro modelo para dar nota ao trabalho do agente (IA como juiz, ou *LLM-as-judge*, capítulo 2) foi considerado e rejeitado naquele contexto. A razão: um juiz que varia avaliando um agente que varia **soma incerteza** (variância, no jargão). Cada número da eval passa a ter duas fontes de ruído em vez de uma.

O critério ficou em código e no gabarito humano.

Isso é ponto de discussão, não regra universal. Há tarefas em que o juiz ajuda: critérios que um teste não pega, como clareza de texto ou qualidade de uma descrição de PR. Nesses casos, meça o juiz também, contra rótulos humanos. Os vieses conhecidos de juízes (preferir a resposta numa certa posição, preferir respostas longas, preferir respostas parecidas com as próprias) estão documentados em Zheng et al. (2023). Para o agente de código, a regra prática é: se dá para verificar com código, verifique com código.

7. Falha em produção vira caso de eval

"O caso que surpreendeu na terça é um teste na quarta." Toda falha observada em produção vira um caso novo no conjunto de eval.

E um aprendizado só vira regra da casa (no prompt, na documentação, num hook) depois de aparecer em **dois ou mais** casos. Isso evita transformar uma exceção em regra para todo mundo.

No repositório, é o ciclo "falha vira tarefa nova" do capítulo 6.

Armadilhas

Cada armadilha abaixo produz um número que parece certo e não é. Todas são fáceis de cair e difíceis de perceber depois. Você não precisa ter lido os capítulos técnicos para entender: cada uma diz o que acontece, por que engana e como evitar. Os detalhes de como o repositório se protege ficam nos capítulos indicados.

1. Uma execução só

O que acontece: você roda cada variante uma vez em cada tarefa, B passa em mais tarefas, e você conclui que B é melhor.

Por que engana: o agente não faz a mesma coisa duas vezes (capítulo 1). Uma execução que deu certo combina com uma taxa real de 21% a 100%. A diferença entre as variantes pode ser só sorte, como tirar cara na primeira jogada da moeda.

Como evitar: repita cada tarefa (o repositório usa 2 repetições), leia a taxa sempre com a margem de erro (intervalo de Wilson) e olhe o "Resolve sempre?" (pass^k). Antes de concluir, veja quantas execuções seriam precisas para saber (capítulo 5).

2. Configuração global vazando

O que acontece: a variante A deveria rodar sem `CLAUDE.md`, mas o Claude Code carrega a sua configuração pessoal: o seu `~/.claude/CLAUDE.md`, os seus hooks e as suas skills globais (capítulo 6 explica hook e skill).

Por que engana: A passa a ter um `CLAUDE.md` escondido. As duas variantes ficam mais parecidas do que você pensa, e a diferença medida encolhe. Pior: o resultado muda de máquina para máquina, conforme a configuração de cada pessoa.

Como evitar: restrinja as fontes de configuração ao projeto (o repositório usa `--setting-sources project`), desligue os conectores da sua conta, que chegam pelo MCP, o protocolo que liga o agente a ferramentas externas (`--strict-mcp-config`, que `--setting-sources` não cobre) e **verifique** na sua máquina: numa cópia da variante A, pergunte ao agente que instruções de memória ele recebeu (capítulo 4). Configuração gerenciada pela organização não sai com flag nenhuma; ela chega igual às duas variantes, então não distorce a comparação, mas o resultado não é "configuração zero".

3. Teste visível ao agente

O que acontece: o teste que define "feito" está no repositório desde o começo, e o agente o lê, ou o edita até passar.

Por que engana: é a prova com o gabarito à vista. O agente otimiza para o teste, não para a tarefa. Ele pode escrever código que só satisfaz aquele caso, ou simplesmente mudar a asserção. A taxa sobe sem o agente ficar melhor.

Como evitar: o teste escondido só entra na pasta **depois** que o agente termina, sobrescreve qualquer arquivo de mesmo nome e roda com uma configuração da ferramenta de testes (o Vitest) que fica fora da cópia (capítulo 3).

4. Tarefa que já passa

O que acontece: o teste escondido passa no código intacto, sem o agente fazer nada. Ou o contrário: nem a solução de referência passa.

Por que engana: uma tarefa que já passa dá sucesso para as duas variantes; uma tarefa impossível dá falha para as duas. Nos dois casos, a tarefa só dilui a diferença e faz o conjunto parecer maior do que é.

Como evitar: verifique toda tarefa automaticamente: ela precisa **falhar** no app intacto e **passar** com a solução de referência. O repositório faz isso para as 8 tarefas, na camada determinística.

5. Contar erro de agente como falha

O que acontece: a rede caiu, o login expirou ou a execução estourou o limite de turnos, e isso entrou na taxa como "falhou".

Por que engana: o erro não diz nada sobre a configuração que você está medindo. É como contar a falta de um aluno doente como nota zero na prova. Se ele se concentrar numa variante (por exemplo, a que rodou durante a queda da rede), essa variante parece pior sem ser.

Como evitar: tenha um desfecho próprio (`erro_agente`), tire-o da conta da taxa e mostre a contagem à parte. Se uma variante tiver muitos erros, rode de novo antes de concluir (capítulo 2).

6. Comparar mudando duas coisas

O que acontece: entre a linha de base e a nova rodada, você mudou o modelo **e** o `CLAUDE.md`. Ou atualizou a versão do Claude Code no meio.

Por que engana: se o resultado mudou, você não sabe quem mudou. Uma das mudanças pode ter ajudado muito e a outra, atrapalhado.

Como evitar: uma mudança por vez (capítulo 6). Fixe tudo o que não está sendo medido: modelo, versão da ferramenta, tarefas, número de repetições. Commite a configuração junto com o resultado.

Mais duas, de bônus

Ler o custo como fatura. O `total_cost_usd` do Claude Code é uma estimativa do lado do cliente. Com assinatura, use como medida de esforço para comparar variantes, não como o valor que você pagou.

Tratar "ainda não dá para saber" como empate. "Ainda não dá para saber" (inconclusivo) quer dizer "não tenho dado para dizer", não "são iguais". As faixas da margem de erro se cruzam; ainda não se separaram. A ação certa é rodar mais, ou aceitar que a diferença, se existe, é pequena demais para valer o esforço (capítulo 5).

Glossário

Na coluna da esquerda, o nome que a palestra usa. No meio, o termo técnico, que é o que você vai achar nos livros, na documentação e no buscador. À direita, uma frase.

O que se mede

Na palestra	Termo técnico	Em uma frase
eval	eval, *evaluation*	Tarefas com critério de "deu certo" conferido por código, repetidas, para comparar antes × depois de uma mudança.
deu certo	sucesso, critério de feito	A regra objetiva que diz se a tarefa foi resolvida; aqui, o teste escondido passar.
teste escondido	teste oculto, *hidden test*	O teste automático que confere a tarefa e que o agente nunca vê: o gabarito da prova.
voltas	turnos, *turns*	Cada ida e volta entre o modelo e uma ferramenta (ler, editar, rodar os testes).
mexeu onde não devia	fora do escopo	Alterou um arquivo que a tarefa não permitia; não conta como sucesso, mesmo que o teste passe.
resolve sempre?	consistência, pass^k	Quantas tarefas deram certo em todas as k tentativas.
acertou ao menos uma vez	pass@k	A tarefa deu certo em pelo menos uma de k tentativas; útil quando uma pessoa escolhe a melhor.
erro do agente	`erro_agente`	O agente nem terminou (internet, login, limite); fica fora da conta da taxa.
desfecho	desfecho, rótulo de resultado	O rótulo único de cada execução: `passou`, `falhou`, `fora_do_escopo` ou `erro_agente`.
IA como juiz	LLM-as-judge	Um modelo de IA dá nota ao trabalho de outro; ajuda onde não há teste, mas também varia.

Como se lê o resultado

Na palestra	Termo técnico	Em uma frase
margem de erro	intervalo de confiança (aqui, intervalo de Wilson, 95%)	A faixa onde a taxa real provavelmente está, como na pesquisa eleitoral.

Na palestra	Termo técnico	Em uma frase
as faixas se cruzam	sobreposição de intervalos	Ainda não dá para dizer quem é melhor.
ainda não dá para saber	inconclusivo	Não há dado suficiente para dizer; não é o mesmo que empate.
quantas execuções para saber	tamanho de amostra	Quantas execuções de cada lado seriam precisas para as faixas se separarem.
o piso	teste z de duas proporções, poder estatístico	Uma estimativa clássica, menos exigente, de quantas execuções seriam precisas.
o valor do meio	mediana	O valor do meio quando tudo está em ordem; não é puxado por casos extremos como a média.
portão	*quality gate*	Uma checagem que quebra o build quando o resultado piora de forma clara.

O experimento e as ferramentas

Na palestra	Termo técnico	Em uma frase
agente de código	*coding agent*	Um programa que usa um modelo de IA para ler, editar e testar código, decidindo o próximo passo sozinho.
o agente não faz a mesma coisa duas vezes	não determinístico	Mesmo pedido, resultados diferentes; por isso uma execução é uma amostra.
pedacinho de texto	token	A unidade de texto que o modelo lê e escreve; é a medida de consumo.
regra da casa	convenção	Um combinado do projeto que não dá para adivinhar só lendo o código.
o arquivo com as regras da casa	`CLAUDE.md`	Texto que o Claude Code lê no começo de toda conversa; é contexto, não obrigação.
o que o modelo tem à frente	contexto	Todo o texto que o modelo leva em conta ao responder: o pedido, os arquivos lidos, as instruções.
manual que ele abre se precisar	skill	Instruções sobre um assunto que o agente só carrega quando a tarefa pede.
script que roda a cada ação	hook	Código seu que roda antes ou depois de cada ação do agente e pode bloqueá-la.

Na palestra	Termo técnico	Em uma frase
variante A / variante B	grupo de controle / tratamento, teste A/B	As duas versões comparadas: o antes e o depois de uma mudança.
tarefa de controle	controle	Tarefa que não depende da mudança; se B ganhar nela também, desconfie.
caixa fechada	sandbox	Isolamento imposto pelo sistema: o agente só escreve na cópia e não tem internet.
sem tela	headless, `claude -p`	O agente roda sem interface interativa, recebe o pedido por argumento e devolve JSON.
conectores da conta	MCP, *Model Context Protocol*	O protocolo que liga o agente a ferramentas externas.
quem roda as execuções	promptfoo	Ferramenta de código aberto que repete os casos, roda em paralelo e grava o resultado.
ferramenta de testes	Vitest	O executor de testes automáticos usado no app de exemplo e na correção.
padrão de texto	regex, expressão regular	Uma forma compacta de descrever um padrão, como "só MAIÚSCULAS e sublinhado".
padrão de caminho	glob	Um caminho com curinga: `*` dentro de uma pasta, `**` também nas subpastas.
roda a cada push	CI, integração contínua	O processo que roda os testes automaticamente a cada push ou PR.
pedido de trabalho	issue	A descrição de uma tarefa, como se abre no GitHub ou no Jira.

Referências

Só fontes públicas, todas conferidas em 26/10/2026. As páginas de documentação mudam: se uma flag citada aqui não bater com a sua versão, vale a documentação atual.

promptfoo

- "Javascript Provider": o contrato do provider próprio (`id`, `callApi`). https://www.promptfoo.dev/docs/providers/custom-api/
- "Command line": `promptfoo eval` e as flags `--repeat`, `-j` / `--max-concurrency`, `-n` / `--filter-first-n` e `-o`. https://www.promptfoo.dev/docs/usage/command-line/
- "LLM Rubric": asserção com modelo como juiz (`llm-rubric`). https://www.promptfoo.dev/docs/configuration/expected-outputs/model-graded/llm-rubric/

Claude Code

- "Run Claude Code programmatically": modo headless (`-p`), `--output-format json` e os campos do JSON; o custo é estimativa do lado do cliente. https://code.claude.com/docs/en/headless
- "How Claude remembers your project": `CLAUDE.md` é contexto, não configuração obrigatória; para bloquear uma ação, hook `PreToolUse`. https://code.claude.com/docs/en/memory
- "Extend Claude with skills": a descrição da skill fica no contexto, o corpo entra quando ela é usada. https://code.claude.com/docs/en/skills
- "Hooks reference": hook `PreToolUse` que sai com código 2 bloqueia a chamada. https://code.claude.com/docs/en/hooks

Codex CLI (desafio extra)

- "Non-interactive mode": `codex exec`, `--json`, `--sandbox workspace-write`, `--ignore-user-config`. https://learn.chatgpt.com/docs/non-interactive-mode

Evals de agentes

- Anthropic (2026), "Demystifying evals for AI agents": pass^k como a chance de todas as k tentativas darem certo; avaliadores em código × avaliadores com modelo. https://www.anthropic.com/research/demystifying-evals-for-ai-agents
- Yao et al. (2024), "τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains": propõe a métrica pass^k para medir a confiabilidade do agente em várias tentativas. https://arxiv.org/abs/2406.12045

- Jimenez et al. (2023), "SWE-bench: Can Language Models Resolve Real-World GitHub Issues?": issues reais do GitHub como tarefa para modelos que editam o código, o mesmo formato das tarefas deste repositório. https://arxiv.org/abs/2310.06770
- Zheng et al. (2023), "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena": vieses de posição, de verbosidade e de autopreferência em juízes. https://arxiv.org/abs/2306.05685

Estatística

- Intervalo de Wilson (Wilson, 1927) e por que ele se comporta melhor que o intervalo "p ± z·√(p(1−p)/n)" com n pequeno ou taxa extrema: Wikipedia, "Binomial proportion confidence interval". https://en.wikipedia.org/wiki/Binomial_proportion_confidence_interval

A fórmula de tamanho de amostra do capítulo 5 é a aproximação normal para comparar duas proporções (1,96 para 95% de confiança bilateral, 0,84 para 80% de poder). `[sem fonte]`: não encontrei uma página pública verificada que traga exatamente esta forma.